



# Introduction to Algorithms

## Homework 1 - Solution

Recommended deadline - 2/11

### 1 Peaks Finding

Recall the logarithmic-time version of find-peak. In the recitation, we started to prove its correctness. In particular, we handled the first two splits in the if-else junctions, lines (2)-(7). Complete the proof by treating the last split, namely lines (8)-(10).

**Result:** returns a peak of  $A_1 \dots A_n \in \mathbb{R}$

```
1  $i \leftarrow \lceil n/2 \rceil$ 
2 if  $A_i$  is a peak then
3   | return  $i$ 
4 end
5 else if  $A_{i-1} \leq A_i$  then
6   | return  $i + \text{find-peak}(A_{i+1} \dots A_n)$ 
7 end
8 else
9   | return  $\text{find-peak}(A_1, A_2, A_3 \dots A_{i-1})$ 
10 end
```

**Algorithm 1:** Improved algorithm

**Solution.** Guideline. For completing the induction step, we have to prove a similar claim to 'claim 2', stated as follows. Let  $A = A_1, A_2, \dots, A_{n-1}, A_n$  be a  $n$ -length array. Suppose that  $A_{j-1} > A_j$  and define  $A'$  to be the array obtained by taking the elements  $A_1, A_2 \dots A_{j-1}$ . Then any peak of  $A'$  is also a peak of  $A$ .

### 2 Peaks-Finding on the Cycle

Consider the following variation of the peaks-finding problem. We will define the peaks again to be the local maximas of given array  $A$ . However instead of thinking on position  $1, 2 \dots n$  as coordinates on the line we will think about them as coordinates on the cycle. Namely  $A_1$  is the right neighbor of  $A_n$ .

Show a series of arrays  $\{A_n\}_{n=3}^{\infty}$  for which the recursive version of peak-find always fails. Explain in one line why the naive algorithm is still correct (You don't have to provide a formal correctness proof).

**Solution.** Guideline. For any  $n \geq 3$  define  $A_n$  as follow:

$$A_i = \begin{cases} n & \text{if } i = 1 \\ i - 1 & \text{else} \end{cases}$$

So we have that

$$A_{\lceil n/2 \rceil - 1} = \lceil n/2 \rceil - 2 \leq A_{\lceil n/2 \rceil} = \lceil n/2 \rceil - 1 \leq A_{\lceil n/2 \rceil + 1} = \lceil n/2 \rceil$$

Hence, the algorithm will answer affirmatively on line (5) and will look in recursion on the right half of the array which doesn't contain a peak. Yet the naive version iterates over the whole array, in our case it will check if  $A_1 \geq A_2, A_n$ , answer affirmatively and returns 1. In fact, as the global maximum is a peak it's guaranteed that for any array set on the cycle the naive version returns a peak.

### 3 $k$ -range peaks

In this section we define  $k$ -range peak to be all the items satisfy:

$$A_i = \max_{|i-j| \leq k} \{A_j\}$$

Write an algorithm that find a  $k$ -range peak in at most  $k \log(n)$  up to constant factor independent on  $k$  and  $n$ . Prove correctness, and bound the running time.

**Solution.** Consider Algorithm 2. Let's prove correction by induction.

*Proof.* 1. Base, for  $n \leq k$ . Trivial.

2. Assumption, Assume that for any  $m$ -length array, such that  $m < n$  the alg returns a peak.
3. Step, Consider an array  $A$  of length  $n$ . If  $A_j$  is a  $k$ -range peak, then the algorithm answers affirmatively on the first check and we're done. If not, then either  $i < j$  or  $j < i$ .

Consider the first case,  $i < j \Rightarrow A_{j-k}, A_{j-k-1}, \dots, A_{j-1} \leq A_j$ . Using a variation of Claim 1.3.2 (which we didn't prove, and you had to do so), we have that any  $k$ -range peak of  $A' = A_j, A_{j+1}, \dots, A_n$  is also a  $k$ -range peak of  $A$ . The length of  $A'$  is equal to or less than  $n/2 < n$ . So, by the induction assumption, in the case where  $i < j$ , the algorithm returns a  $k$ -range peak of  $A'$ , denoted by  $l$  its position in  $A'$  and notice that its corresponding position in  $A$  is  $j - 1 + l$ .

In the other case, we have  $j < i$  (otherwise  $j = i$  would be a peak). The second case  $i > j$  is similar. Again, using a variation of Claim 1.3.2,  $A_{j+k}, A_{j+k-1}, \dots, A_{j+1} \leq A_j$  and therefore any  $k$ -range peak of  $A' = A_1, A_2, \dots, A_j$

is also a  $k$ -range peak of  $A$ . And again,  $A'$  has length less than  $n/2$  and hence, by the induction assumption, the algorithm returns a  $k$ -range peak of  $A'$  which is also a  $k$ -range peak of  $A$ .

□

**Result:** returns a  $k$ -range peak of  $A_1 \dots A_n \in \mathbb{R}$

```

1  $i \leftarrow \lceil n/2 \rceil$ 
2  $j \leftarrow \arg \max_{|i-j| \leq k} A_j$ 
3 if  $A_j$  is a  $k$ -range peak then
4   | return  $j$ 
5 end
6 else if  $i < j$  then
7   | return  $j - 1 + \text{find-peak}(A_j, A_{j+1} \dots A_n)$ 
8 end
9 else
10  | return  $\text{find-peak}(A_1, A_2, A_3 \dots A_j)$ 
11 end
```

**Algorithm 2:**  $k$ -range peak-find algorithm

Running time, Observing that in most worst-case scenarios, in any iteration, we observe  $\sim 2k + 1$  elements and then call one of the recursive calls on an array of size at most  $\lceil n/2 \rceil$ . Therefore, the running time  $T(n)$  in that case obeys the following recursive rule:

$$T(n) \leq c \cdot k + T(\lceil n/2 \rceil)$$

Thus, if we denote  $c' = c \cdot k$ , we obtain the same recursive formula we solved in the reaction  $\Rightarrow T(n) = \Theta(\log n)$ .

## 4 2D-peaks

Let  $A$  be a  $n \times n$  matrix. We will say that  $A_{ij}$  is a peak if:

$$A_{ij} \geq A_{i-1,j}, A_{i+1,j}, A_{i,j-1}, A_{i,j+1}$$

If the indices exceed the matrix frame, we define the inequality as satisfied (For example if  $A_{1,1} \geq A_{1,2}, A_{2,1}$  then  $A_{1,1}$  is a peak).

Concurrent examples. Consider the following matrices:

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \quad B = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \quad C = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

The peaks of  $A$  are  $A_{1,1} = 1$  and  $A_{2,2} = 0$ .  $B$  has only a single peak:  $B_{1,2} = 2$ .  $C$  has four peaks:  $C_{1,1} = 1$ ,  $C_{2,2} = 2$ ,  $C_{3,3} = 0$ , and  $C_{1,3} = 0$ .

Write an algorithm that find a peak in  $A$ . Solutions that runs in at most  $cn \log n$  are sufficient. Prove correctness, and bound the running time.

**Solution.** Let  $B$  be a  $n$ -length array defined as  $B_i = \max_j A_{i,j}$ .

**Claim 1.** For any  $l \in [n]$  such that  $B_l$  is a peak in  $B$ , we have that for  $k \leftarrow \arg \max_j A_{l,j}$ , the element  $A_{l,k}$  is a peak in  $A$ .

*Proof.* By definition of  $B$ :

$$\begin{aligned} A_{lk} = B_l &\geq B_{l-1} = \max_j A_{l-1,j} \geq A_{l-1,k} \\ B_l &\geq B_{l+1} = \max_j A_{l+1,j} \geq A_{l+1,k} \\ B_l &\geq A_{l,k+1} \\ B_l &\geq A_{l,k-1} \end{aligned}$$

□

Notice that Claim 1 already gives us an  $O(n^2)$  time algorithm. First compute  $B$  in  $O(n^2)$  time, then call find-peak on  $B$  to return an index  $l$  such that  $B_l$  is a peak in  $B$ , thus it remains to return  $\arg_k \max A_{l,k}$  (which can be done in  $O(n)$  time).

We are ready to present the  $n \log n$  time algorithm. Instead of computing  $B$  at the beginning, we are going to compute  $B_i$  only when we need to access it. That is, any time that find-peak will compare between two elements of  $B$   $B_i, B_j$  we will compute their value. Thus, each access to  $B$  costs  $O(n)$  time. However, as the running time of the original peak-find is at most  $O(\log(n))$ , it follows that the total number of accesses is also bounded by  $O(\log n)$ , so in total the running time is at most  $O(n \log n)$ . The pseudocode is presented in Algorithm 3.

**Result:** returns a peak of  $A_{1,1}, \dots, A_{n,n} \in \mathbb{R}$

- 1  $l \leftarrow \text{Call to find-peak}(B_1, B_2 \dots B_n)$
- 2 when compute  $B_i$  on every accsses.
- 3  $k \leftarrow \arg \max_k A_{l,k}$
- 4 **return**  $l, k$

**Algorithm 3:** 2D peak-find algorithm

**Correctness.** Assuming contradiction, suppose there exists a matrix  $A$  on which Algorithm 3 fails to return a peak. Claim 1 guarantees that if  $l$  is indeed a peak, then  $A_{l,k}$  for  $k = \arg \max_k A_{l,k}$  is a peak in  $B$ . Hence, the algorithm fails only if find-peak of fails to return the position of a peak in  $B$ . In other words, we have found an array  $B$  on which find-peak fails. Thus contradicting the correctness of find-peak.